



PROJECT 1 • HANDS-ON LAB

Deploy an AWS VPC with Private & Public Subnets

Using reusable Terraform modules to provision networking infrastructure on AWS.

VPC • Public & Private Subnets • Internet Gateway • Route Tables

What We're Building

A production-style VPC, built once as a Terraform module and reused anywhere.



Public & Private Subnets

Segmented address space for internet-facing and internal-only resources.



Internet & NAT Gateway

Controlled outbound and inbound internet access per subnet type.



Routing Tables

Explicit routes tying subnets to the right gateway.

Module Layout

A root configuration calls a self-contained VPC module — variables in, resources built, outputs out. This is the pattern you'll reuse across every future project.

STEP 1

Create a Terraform Module for VPC

A clean module layout separates reusable logic (modules/vpc) from the root config that calls it.

terraform-vpc/

 main.tf

modules/

vpc/

 main.tf

 variables.tf

 outputs.tf



modules/vpc/

Self-contained module: defines every VPC resource once.



variables.tf

Declares the inputs the module accepts (CIDR blocks, names).



Root main.tf

Calls the module and supplies real values for this environment.

STEP 2

Define the VPC Module

modules/vpc/main.tf — VPC and subnets

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
  tags = {
    Name = var.vpc_name
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet_cidr
  map_public_ip_on_launch = true
}

resource "aws_subnet" "private_subnet" {
  vpc_id      = aws_vpc.main.id
  cidr_block  = var.private_subnet_cidr
}
```



aws_vpc.main

The container network. CIDR and name come from variables.



public_subnet

map_public_ip_on_launch = true auto-assigns public IPs.



private_subnet

No public IP mapping — stays internal by default.

STEP 2

Define the VPC Module (continued)

modules/vpc/main.tf — gateway and routing

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route" "public_route" {
  route_table_id      = aws_route_table.public_rt.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id          = aws_internet_gateway.igw.id
}

resource "aws_route_table_association" "public_assoc" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}
```



internet_gateway

Attached to the VPC — the door to the public internet.



0.0.0.0/0 → igw

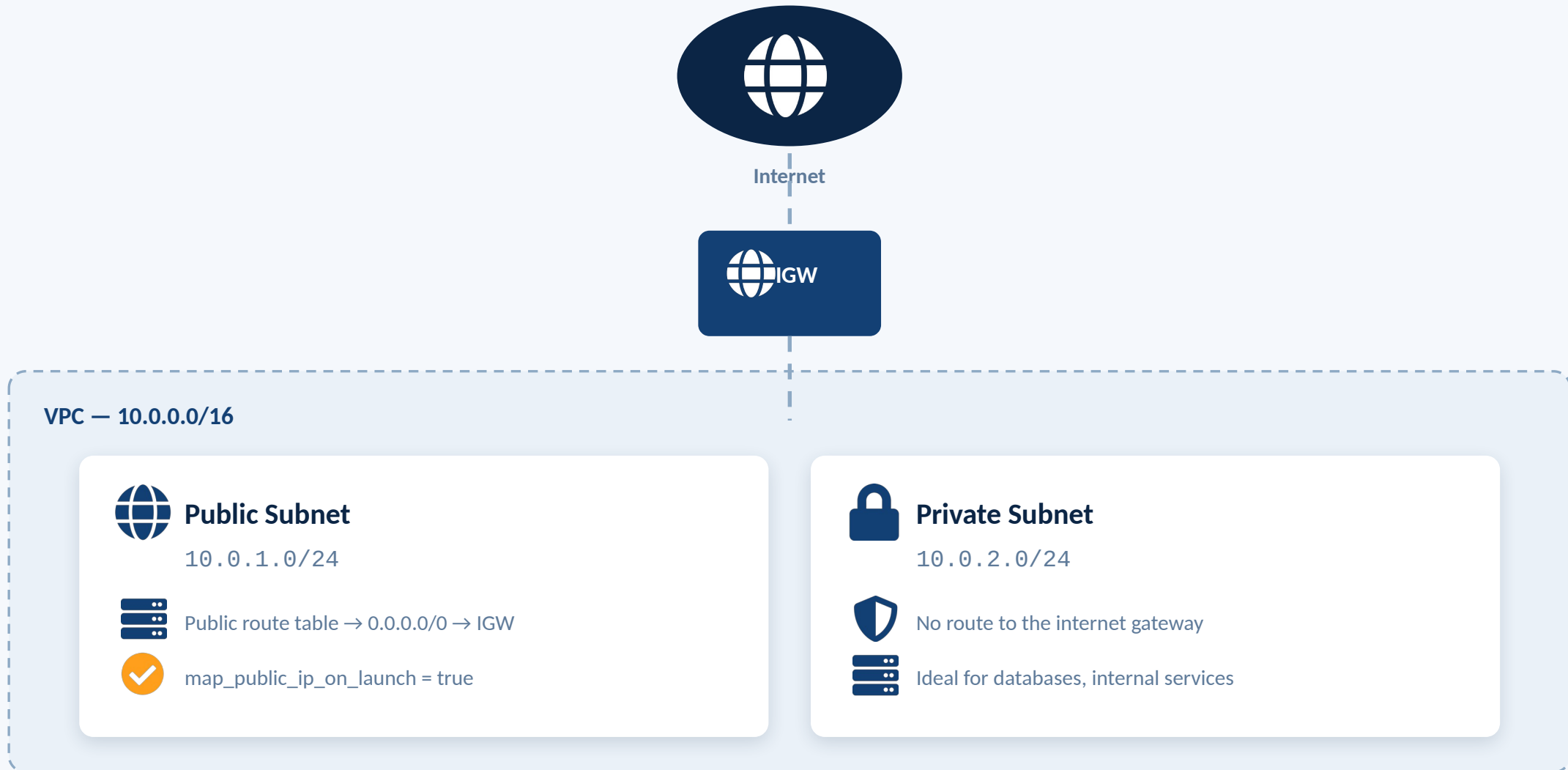
Any destination routes out through the gateway.



route_table_association

Binds the public subnet to this route table.

How It All Connects



Variables & Calling the Module

Step 3 — modules/vpc/variables.tf

```
variable "vpc_cidr" {}  
variable "vpc_name" {}  
variable "public_subnet_cidr" {}  
variable "private_subnet_cidr" {}
```

Step 4 — root main.tf

```
module "vpc" {  
  source = "../modules/vpc"  
  
  vpc_cidr           = "10.0.0.0/16"  
  vpc_name           = "MyVPC"  
  public_subnet_cidr = "10.0.1.0/24"  
  private_subnet_cidr = "10.0.2.0/24"  
}
```



Empty variable blocks mean no defaults — every value must be supplied by the caller, keeping the module fully reusable.

Now run Terraform to create the VPC:

```
$ terraform init  
$ terraform apply
```

Initialize, Validate & Preview

1



Initialize Terraform

```
$ terraform init
```

Downloads required providers and initializes the working directory.

2



Validate the Configuration

```
$ terraform validate
```

Checks for syntax errors. Returns:
Success! The configuration is valid.

3



Preview the Changes

```
$ terraform plan
```

Lists all VPC, subnet, IGW, and route table resources to be created.

Apply the Configuration

Step 4 — If everything looks good, apply the changes.

```
$ terraform apply -auto-approve
```

Terraform will create:

- ✓ VPC with CIDR block 10.0.0.0/16
- ✓ Public subnet 10.0.1.0/24
- ✓ Private subnet 10.0.2.0/24
- ✓ Internet Gateway for public subnet
- ✓ Route tables & associations

Expected output:

```
Apply complete!  
Resources: 5 added, 0 changed,  
0 destroyed.
```



5 Resources Created

VPC · Public Subnet · Private Subnet · Internet Gateway · Route Table

Verify & Clean Up



Step 5 — Verify in AWS Console

Go to AWS Console → VPC Dashboard and check:



VPCs

Verify that “MyVPC” exists.



Subnets

Check the public and private subnets.



Route Tables

Ensure the public subnet has an Internet Gateway.



Step 6 — Clean Up (Optional)

If you want to destroy everything:

```
$ terraform destroy  
-auto-approve
```

This removes all resources created by Terraform — the VPC, both subnets, the internet gateway, and route tables.

Use only in sandbox / learning accounts

PROJECT 1 SUMMARY

What You Built



A Reusable VPC Module

Encapsulated VPC, subnets, gateway, and routing behind clean variables.



Public & Private Segmentation

Public subnet routes to the internet; private subnet stays isolated.



A Repeatable Workflow

init → validate → plan → apply → verify → destroy.